Щучкин Максим, Б07-202

Отчет по лабораторной работе #2

«Реализация задачи численного интегрирования по гибридной технологии»

Цель: освоить гибридную модель параллелизма, объединяющую тяжелые процессы (fork) и легкие потоки (pthreads) в одной программе, решающей задачу численного интегрирования.

Текст программы:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <pthread.h>
#include <omp.h>
#include <errno.h>

#define MSG_ID 7777
#define MSG_PERM 0666
#define MAX_NI 1000000000

// Структура сообщения для IPC
typedef struct { long mtype; double sum; } msg_t;

/* Параметры для потока */
typedef struct { int tid; double a, b; int n; double result; } thread_data_t;

/* Интегрируемая функция */
double f1(double x) { return 4.0 / (1.0 + x * x); }

/* Метод средних прямоугольников */
double integrate(double (*f)(double), double a, double b, int n) {
    double h = (b - a) / n;
    double s = 0.0;
    for (int i = 0; i < n; i++) s += f(a + h * (i + 0.5));
    return h * s;
}

void* thread_func(void* arg) {
    thread_data_t* td = (thread_data_t*)arg;
    td->result = integrate(f1, td->a, td->b, td->n);
    return NULL;
```

```
}

int main(int argc, char* argv[]) {
    setbuf(stderr, NULL); // Мгновенный вывод без буферизации

    int np = 1, nt = 1;
    if (argc >= 3) { np = atoi(argv[1]); nt = atoi(argv[2]); }
    if (np < 1) np = 1;
    if (nt < 1) nt = 1;

    double t_start = omp_get_wtime();

    // Очередь создается ДО ветвления, чтобы все процессы получили один msgid
    int msgid = msgget(MSG_ID, IPC_CREAT | MSG_PERM);
    if (msgid < 0) { perror("msgget"); exit(1); }

    int mp = 0;
    pid_t spid;

    // создание np-1 дочерних процессов
    for (int i = 1; i < np; i++) {
        spid = fork();
        if (spid < 0) { perror("fork"); exit(1); }
        if (spid == 0) {
            mp = i;
            break;
        }
    }

    // Распределение области интегрирования
    double h_proc = 1.0 / np;
    double a_proc = h_proc * mp;
    double b_proc = (mp == np - 1) ? 1.0 : a_proc + h_proc;
    double h_th = (b_proc - a_proc) / nt;
    int n_th = MAX_NI / (np * nt);

    pthread_t threads[nt];
    thread_data_t tdata[nt];

    // Создание и запуск потоков
    for (int t = 0; t < nt; t++) {
        tdata[t].tid = t;
        tdata[t].a = a_proc + h_th * t;
        tdata[t].b = (t == nt - 1) ? b_proc : a_proc + h_th * (t + 1);
        tdata[t].n = n_th;
        pthread_create(&threads[t], NULL, thread_func, &tdata[t]);
    }
```

```
    // Суммирование
    double proc_sum = 0.0;
    for (int t = 0; t < nt; t++) {
        pthread_join(threads[t], NULL);
        proc_sum += tdata[t].result;
        fprintf(stderr, "mp=%d mt=%d a1=%le b1=%le n1=%d s1=%le\n",
                mp, t, tdata[t].a, tdata[t].b, tdata[t].n, tdata[t].result);
    }

    //  Обмен через очередь сообщений System V
    if (mp == 0) {
        for (int i = 1; i < np; i++) {
            msg_t msg;
            msgrcv(msgid, &msg, sizeof(double), 0, 0);
            proc_sum += msg.sum;
        }
    } else {
        msg_t msg;
        msg.mtype = mp;
        msg.sum = proc_sum;
        msgsnd(msgid, &msg, sizeof(double), 0);
    }

    double t_elapsed = omp_get_wtime() - t_start;

    if (mp == 0) {
        printf("time=%lf sum= %le\n", t_elapsed, proc_sum);
        msgctl(msgid, IPC_RMID, NULL);
    }

    return 0;
}
```

Output:

```
fnbik09@sr210:~/mpc/task2$ gcc -O2 -pthread solution_02.c -o solution_02.px -lm -fopenmp
fnbik09@sr210:~/mpc/task2$ ./solution_02.px 1 1
mp=0 mt=0 a1=0.000000e+00 b1=1.000000e+00 n1=1000000000 s1=3.141593e+00
time=1.732338 sum= 3.141593e+00
fnbik09@sr210:~/mpc/task2$ ./solution_02.px 2 2
mp=0 mt=0 a1=0.000000e+00 b1=2.500000e-01 n1=250000000 s1=9.799147e-01
mp=0 mt=1 a1=2.500000e-01 b1=5.000000e-01 n1=250000000 s1=8.746758e-01
mp=1 mt=0 a1=5.000000e-01 b1=7.500000e-01 n1=250000000 s1=7.194140e-01
mp=1 mt=1 a1=7.500000e-01 b1=1.000000e+00 n1=250000000 s1=5.675882e-01
time=0.518603 sum= 3.141593e+00
fnbik09@sr210:~/mpc/task2$ ./solution_02.px 4 2
mp=2 mt=0 a1=5.000000e-01 b1=6.250000e-01 n1=125000000 s1=3.798068e-01
mp=2 mt=1 a1=6.250000e-01 b1=7.500000e-01 n1=125000000 s1=3.396072e-01
mp=0 mt=0 a1=0.000000e+00 b1=1.250000e-01 n1=125000000 s1=4.974200e-01
mp=0 mt=1 a1=1.250000e-01 b1=2.500000e-01 n1=125000000 s1=4.824947e-01
mp=1 mt=0 a1=2.500000e-01 b1=3.750000e-01 n1=125000000 s1=4.551680e-01
mp=1 mt=1 a1=3.750000e-01 b1=5.000000e-01 n1=125000000 s1=4.195078e-01
mp=3 mt=0 a1=7.500000e-01 b1=8.750000e-01 n1=125000000 s1=3.013156e-01
mp=3 mt=1 a1=8.750000e-01 b1=1.000000e+00 n1=125000000 s1=2.662727e-01
time=0.260624 sum= 3.141593e+00
fnbik09@sr210:~/mpc/task2$
```

Выводы:

1) Вывод программы корректен, время выполнения уменьшается с ростом числа процессов/потоков, итоговая сумма 3.141593e+00 совпадает с аналитическим значением интеграла.
2) Строки mp=X mt=Y... выводятся в произвольном порядке. Это ожидаемое поведение: планировщик ОС не гарантирует детерминированный порядок завершения потоков/процессов без явных барьеров. Корректность результата гарантируется математической независимостью подинтервалов и атомарностью операции сложения на уровне double в рамках одного процесса перед отправкой.